Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет информационных технологий и управления

Кафедра вычислительных методов и программирования

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему:

«СИСТЕМА УЧЕТА РАСХОДОВ НА РЕМОНТ И ОБСЛУЖИВАНИЕ АВТОМОБИЛЕЙ»

Выполнил: гр.421701 Воинов В.С.

Проверила: Новицкая Л.И.

Минск 2025

РЕФЕРАТ

ТЕМА: СИСТЕМА УЧЕТА РАСХОДОВ НА РЕМОНТ И ОБСЛУЖИВАНИЕ АВТОМОБИЛЕЙ / В.С. Воинов. Минск БГУИР, 2025, - п.з. – 52с., рисунков - 9, источников - 3, приложений – 2.

Цель курсовой работы: разработка программного обеспечения, обеспечивающего учёт расходов на техническое обслуживание и ремонт автомобилей с использованием бинарных файлов и ручных алгоритмов обработки.

Методология проведения работы: в процессе разработки была реализована консольная система на языке программирования C++ в среде MS Visual Studio 2022. Программа использует структурированный подход к организации данных с применением пользовательских структур и бинарных файлов. Разработанная система поддерживает добавление, удаление, редактирование, поиск и сортировку записей, а также автоматическое формирование текстового отчёта о действиях пользователя.

Результаты работы: описана структура хранения информации о ремонтных работах; реализованы алгоритмы сортировки и поиска; предусмотрена система логирования. Интерфейс построен на многоуровневом меню с проверками корректности ввода. Программа успешно протестирована на различных сценариях и демонстрирует стабильную работу с бинарными файлами.

Область применения результатов: разработанная система может быть использована как в учебных целях, так и в качестве базового прототипа для автоматизации учёта в малых автосервисах и технических отделах.

СОДЕРЖАНИЕ

Введение 6
1 Структуры и файлы 7
1.1 Структуры данных в языке C/C++ 7
1.2 Работа с файлами в C/C++ 8
2 Алгоритмы сортировки 10
2.1 Описание и обоснование выбора алгоритмов сортировки 10
2.2 Быстрая сортировка (QuickSort) 10
2.3 Сортировка выбором (Selection Sort) 11
2.4 Сортировка вставками (Insertion Sort) 12
2.5 Сравнительный анализ эффективности алгоритмов 12
3 Алгоритмы поиска 14
3.1 Линейный поиск по модели автомобиля алгоритмов 14
3.2 Бинарный поиск по стоимости ремонта 14
3.3 Поиск по признаку (марка и максимальная стоимость ремонта) 15
3.4 Итоги по алгоритмам поиска 16
4 Пользовательские функции 17
4.1 Функции пользовательского интерфейса и навигации 17
4.2 Функции работы с записями 17
4.3 Алгоритмы поиска и сортировки 19
4.4 Вспомогательные функции для работы с файлами 19
4.5 Функции валидации и ввода 20
4.6 Отладочные функции 21
4.7 Итог реализации пользовательских функций 22
5 Описание работы программы 23
Заключение 27
Список использованных источников 28
Приложение А (обязательное) Листинг кода 29
Приложение Б (обязательное) Блок-схема работы программы 51

ВВЕДЕНИЕ

Современные информационные технологии играют ключевую роль в эффективной организации и управлении бизнес-процессами. Одной из актуальных задач в этой области является автоматизация учета и анализа данных о техническом обслуживании и ремонте автомобилей. Такая задача особенно важна для автосервисов и ремонтных мастерских, где требуется вести систематизированный учет предоставленных услуг, их стоимости, типов ремонтов и технических характеристик обслуживаемых транспортных средств.

Объектом исследования данной работы является процесс хранения и обработки данных о ремонтах автомобилей в программной среде. В качестве предмета исследования рассматриваются алгоритмы обработки этих данных, включая сортировку, поиск и интерфейсные решения, направленные на упрощение взаимодействия с системой.

Цель курсовой работы — разработка программного средства, позволяющего вести учет ремонтных записей автомобилей, с возможностью их сортировки, поиска и статистической обработки. Программа должна обеспечивать удобный интерфейс взаимодействия с пользователем, хранение данных в файле, корректный ввод информации и автоматическую генерацию отчета.

Для достижения поставленной цели необходимо решить следующие задачи:

– реализовать структуру хранения данных о ремонтах автомобилей, включающую основные параметры: марка, модель, тип ремонта, стоимость, пробег и дата;

– обеспечить сохранение и загрузку данных из файла;

– реализовать алгоритмы сортировки по дате ремонта, пробегу и стоимости;

– реализовать алгоритмы поиска: линейного и двоичного;

– реализовать функцию поиска по признаку: марка автомобиля и максимальная стоимость ремонта;

– реализовать отображение статистики по типам ремонта;

– обеспечить выявление самого дорогого и самого дешевого ремонта;

– предоставить пользователю функции добавления, редактирования, удаления записей, а также отображения информации и статистики;

– реализовать формирование отчета с результатами операций.

Актуальность работы обусловлена широким распространением автотранспортных средств и необходимостью автоматизации процессов учета и анализа данных в автосервисах. Использование алгоритмов сортировки и поиска позволяет повысить скорость обработки информации, а интуитивно понятный интерфейс делает систему доступной для пользователей с различным уровнем подготовки.

1 СТРУКТУРЫ И ФАЙЛЫ

1.1 Структуры данных в языке C/C++

Структура представляет собой пользовательский тип данных, объединяющий несколько переменных разного типа под одним именем. В языке C/C++ структуры используются для логической группировки информации, которая относится к одному объекту. Такие переменные называются поля структуры, а сама структура — составной тип данных[3].

В программной реализации работы была использована структура данных `RepairRecord`, инкапсулирующая сведения о ремонтных работах, проводимых с транспортными средствами. Структура включает в себя следующие поля:

– `brand` — строка символов, представляющая марку автомобиля (тип `char[25]`);

– `model` — строка символов, представляющая модель автомобиля (тип `char[25]`);

– `repairType` — перечисление (`enum class RepairType`), определяющее тип ремонта (ТО, кузовной ремонт и др.);

– `repairCost` — целое число, определяющее стоимость ремонта (тип `int`);

– `mileage` — пробег автомобиля на момент ремонта (тип `int`);

– `repairDate` — составной тип `RepairDate`, включающий день, месяц и год ремонта (три целых поля: `day`, `month`, `year`).

```
class RepairRecord {
public:
    char brand[25]{};
    char model[25]{};
    RepairType repairType;
    int repairCost;
    int mileage;
    RepairDate repairDate;
```

Рисунок 1.1 – Поля класса RepairRecord

Использование перечисления позволяет сократить количество ошибок при вводе данных и облегчает фильтрацию и анализ записей по типу ремонта. Значения перечисления жёстко заданы в коде, что исключает ввод некорректных данных и повышает надёжность программы. При отображении или анализе данных достаточно сравнить тип ремонта с конкретным элементом перечисления, что упрощает реализацию сортировки, группировки и статистических операций.

Текстовые поля `brand` и `model` представлены массивами символов фиксированной длины `char[25]`. Это решение обусловлено необходимостью работы с бинарными файлами, где важно сохранять одинаковый размер каждой записи. Благодаря этому каждая запись в бинарном файле занимает одинаковое количество байт, что делает возможным произвольный доступ к данным по смещению без необходимости построчного чтения всего файла. Такой подход упрощает реализацию операций поиска, сортировки и редактирования записей, а также повышает общую производительность программы при работе с файлами большого объёма. Однородная структура данных облегчает чтение и запись информации на уровне побайтового копирования, без дополнительной обработки.

1.2 Работа с файлами в C/C++

Файлы являются ключевым элементом для хранения данных между запусками программы. В данной работе используется бинарный файл, обеспечивающий быстрое и компактное сохранение информации[2]. Формат файла: последовательность объектов `RepairRecord`, записанных в память напрямую.

Каждая запись имеет фиксированный размер — 88 байт, благодаря чему возможен произвольный доступ к любой записи по смещению. Это исключает необходимость построчного чтения и ускоряет выполнение операций поиска, сортировки и редактирования.

Обработка файлов реализована средствами языка C, с использованием функций `fopen`, `fread`, `fwrite`, `fclose`. Пример записи записи в файл:

```
fwrite(&record, sizeof(RepairRecord), 1, file);
```

Чтение записи из файла:

```
fread(&record, sizeof(RepairRecord), 1, file);
```

Файл открывается в режиме:
- `"rb"` — чтение;
- `"wb"` — перезапись;
- `"ab"` — добавление;
- `"rb+"` — чтение и запись.

Имя файла задаётся пользователем при запуске, например:

```
CarRepairSystem system("data.bin");
```

Особенности реализации:

– Добавление записи: открытие файла в режиме `"ab"`, ввод данных через консоль, формирование объекта `RepairRecord`, запись в файл, логирование действия;

– Чтение всех записей: открытие файла в режиме `"rb"`, последовательное чтение всех записей с помощью `fread`, вывод каждой через `printRepairRecord`;

– Удаление/редактирование: файл переписывается построчно во временный файл (`temp.bin`), далее оригинальный файл удаляется и временный переименовывается. Такая стратегия повышает надёжность — если произойдёт сбой, исходный файл не будет повреждён.

Все действия пользователя логируются в отдельный текстовый файл `report.txt`. Каждая операция сопровождается временной меткой, содержанием действия и подробным описанием записи. Это упрощает контроль и отладку системы.

```
=== Report added on: Wed May 22 16:01:20 2025
Added record:
BMW X5, Maintenance, $250, 123456km, 12.05.2023
```

Рисунок 1.2 – Пример записи в лог

Подытоживая, спользование структур и бинарных файлов позволяет создать надёжную, быструю и компактную систему хранения информации. Фиксированный размер записей обеспечивает эффективный произвольный доступ, что упрощает реализацию алгоритмов сортировки, поиска и редактирования данных. Применение объектно-ориентированного подхода, логирования пользовательских действий, а также модульной архитектуры в рамках класса `CarRepairSystem` делает систему устойчивой к изменениям и удобной для дальнейшего расширения функциональности.

2 АЛГОРИТМЫ СОРТИРОВКИ

2.1 Описание и обоснование выбора алгоритмов сортировки

В курсовой работе реализованы три различных алгоритма сортировки, каждый из которых применяется к определённому полю структуры `RepairRecord`:

– быстрая сортировка (`QuickSort`) — используется для сортировки по дате ремонта;

– сортировка выбором (Selection Sort) — применяется для сортировки по пробегу автомобиля;

– сортировка вставками (Insertion Sort) — применяется для сортировки по стоимости ремонта.

Выбор данных алгоритмов обусловлен несколькими ключевыми факторами. Прежде всего, важно было продемонстрировать, как различные подходы к сортировке могут быть адаптированы под конкретные задачи в рамках структуры `RepairRecord`, где каждое поле требует индивидуального подхода.

Быстрая сортировка была выбрана как наиболее производительный алгоритм для работы с полем `repairDate`, поскольку она показывает отличные результаты на больших массивах данных и может использоваться в реальных системах, где скорость критична. Кроме того, её реализация позволяет применять более сложную логику сравнения, что идеально подходит для сравнения составных типов, таких как дата.

Сортировка выбором и сортировка вставками, несмотря на свою асимптотически худшую производительность, имеют важную учебную ценность. Они легко реализуются, наглядны для анализа и позволяют закрепить понимание базовых механизмов сортировки. Сортировка выбором хорошо иллюстрирует идею поиска экстремума и обмена, а сортировка вставками показывает, как можно оптимизировать работу алгоритма на почти отсортированных данных.

В итоге, совокупность выбранных методов позволяет не только решить поставленные задачи, но и провести сравнительный анализ, выявить сильные и слабые стороны каждого из подходов и сделать обоснованный выбор алгоритма под конкретную ситуацию.

2.2 Быстрая сортировка (QuickSort)

Быстрая сортировка реализует стратегию "разделяй и властвуй", при которой исходный массив последовательно разбивается на подмассивы на основе выбранного опорного элемента (пивота). В наиболее распространённой реализации пивот выбирается как срединный элемент, однако также возможно использование других подходов — от фиксированных позиций (первый, последний) до продвинутых методов, например, медианы трёх или случайного

выбора. Это может повлиять как на производительность, так и на устойчивость алгоритма к заранее отсортированным или наоборот — сильно перемешанным данным.

На этапе разбиения (partition) производится последовательное сравнение каждого элемента с опорным и его перемещение в соответствующую часть массива. В результате образуются два подмассива, каждый из которых в дальнейшем обрабатывается рекурсивно. Такое деление продолжается до тех пор, пока массивы не станут содержать менее двух элементов.

Достоинство алгоритма заключается в его способности эффективно работать без необходимости выделения дополнительной памяти: все операции производятся непосредственно внутри исходного массива. Это преимущество отличает быструю сортировку от, например, сортировки слиянием. Впрочем, стоит учитывать, что быстрая сортировка не является стабильной: при наличии одинаковых элементов их порядок может измениться. Это связано с тем, что алгоритм производит обмены независимо от первоначального положения элементов. Таким образом, при наличии повторяющихся значений нельзя гарантировать сохранение их исходного порядка в итоговой выборке. В реальных приложениях для повышения стабильности и устойчивости алгоритм может быть модифицирован, включая использование хвостовой рекурсии, предварительную рандомизацию и другие техники оптимизации.

В рамках данной работы быстрая сортировка используется для упорядочивания записей по дате ремонта, представленным в виде составной структуры `repairDate`. Для корректного сравнения дат были реализованы перегруженные операторы сравнения < и >, что позволяет удобно и наглядно использовать алгоритм без дополнительной логики.

Алгоритм демонстрирует высокую производительность и является одним из самых эффективных универсальных методов сортировки. В среднем его временная сложность составляет $O(n \log n)$, что делает его предпочтительным при обработке больших объёмов неупорядоченных данных. Однако стоит учитывать, что при неудачном выборе опорного элемента возможна деградация производительности до $O(n^2)$, особенно если данные уже отсортированы[1].

2.3 Сортировка выбором (Selection Sort)

Сортировка выбором — один из базовых алгоритмов сортировки, известный своей простотой и наглядностью. Её суть заключается в последовательном проходе по массиву: на каждой итерации выбирается минимальный (или максимальный — в зависимости от порядка сортировки) элемент из неотсортированной части массива и перемещается в первую позицию этой части. Тем самым формируется всё возрастающая отсортированная часть, пока не будут упорядочены все элементы[1].

Алгоритм не требует дополнительной памяти, так как все операции выполняются в пределах исходного массива. В этом заключается его преимущество по сравнению с некоторыми другими методами. Также он легко

реализуется, хорошо подходит для начального изучения алгоритмов сортировки и часто используется в учебных целях.

В рамках работы сортировка выбором применяется для упорядочивания записей по полю `mileage` — пробегу автомобиля на момент ремонта. Это позволяет быстро сформировать, например, перечень автомобилей с наименьшим пробегом. Однако из-за квадратичной сложности `O(n^2)` данный метод не подходит для массивов большого размера: при росте числа записей производительность алгоритма значительно падает. Тем не менее, его использование оправдано при небольших объёмах данных или в задачах, где важна именно простота реализации, а не скорость.

2.4 Сортировка вставками (Insertion Sort)

Сортировка вставками — это простой, но эффективный алгоритм, принцип работы которого напоминает процесс вставки новых элементов в отсортированную колоду карт. Каждый очередной элемент сравнивается с предыдущими, и, если необходимо, сдвигается вправо для освобождения места в нужной позиции. Таким образом, формируется постепенно расширяющаяся отсортированная часть массива.

Главным достоинством данного метода является его стабильность: одинаковые элементы сохраняют исходный порядок, что важно в задачах, где сортировка происходит по дополнительному признаку. Благодаря своей предсказуемости и понятности алгоритм хорошо подходит для начального обучения и демонстрации работы со структурами данных.

В данной работе алгоритм применяется к полю `repairCost` и используется для упорядочивания записей по стоимости ремонта. Это удобно, например, при необходимости быстро определить записи с наименьшими или наибольшими затратами. Также стоит отметить, что данный алгоритм особенно эффективен при сортировке почти отсортированных массивов — в таком случае его сложность приближается к `O(n)`. Однако при неупорядоченных данных или при убывающей последовательности производительность алгоритма значительно снижается, достигая сложности $O(n^2)$. Несмотря на это, сортировка вставками остаётся надёжным выбором для массивов небольшого размера, а также используется как вспомогательная стадия в более сложных гибридных алгоритмах, где она применяется на коротких участках данных из-за своей эффективности в малых объёмах.

2.5 Сравнительный анализ эффективности алгоритмов

оценки эффективности каждого метода необходимо учитывать не только теоретическую сложность, но и практическую применимость в контексте конкретных полей структуры RepairRecord. Следует помнить, что алгоритмы могут демонстрировать разные характеристики в зависимости от размера входных данных, их порядка и специфики работы с полями структур.

Ниже представлена таблица, в которой сведены ключевые особенности каждого из реализованных методов сортировки, что позволяет провести обоснованный выбор алгоритма для конкретной задачи.

Таблица 2.1 – Сравнение алгоритмов сортировки

Алгоритм	Поле сортировки	Сложность (средняя)	Особенности применения
Быстрая сортировка	repairDate	O(n log n)	Эффективен на больших массивах
Сортировка выбором	mileage	$O(n^2)$	Прост в реализация, но медленный
Сортировка вставками	repairCost	$O(n^2)$	Быстр при почти отсортированных данных

Таким образом, каждый из реализованных алгоритмов обладает своей областью применения и демонстрирует различные подходы к решению задачи сортировки. Комбинация методов позволяет эффективно обрабатывать массивы структур в зависимости от размера и структуры данных.

3 АЛГОРИТМЫ ПОИСКА

3.1 Линейный поиск по модели автомобиля алгоритмов

Линейный поиск представляет собой простой алгоритм, при котором каждый элемент последовательно сравнивается с заданным ключом. Этот метод не требует предварительной сортировки массива и может применяться ко всем типам неупорядоченных данных. В рамках курсовой работы он реализован в методе `linearSearchByModel(const char* model)` и используется для поиска записей, соответствующих заданной модели автомобиля.

Суть алгоритма заключается в последовательном чтении всех записей из бинарного файла. Каждая запись временно загружается в оперативную память и сравнивается по значению поля `model`. Если совпадение обнаружено, запись выводится на экран и одновременно сохраняется в текстовый отчёт.

Пример внутренней логики:

```
if (strcmp(tempRecord.model, model) == 0) {
     // Совпадение найдено
     tempRecord.printRepairRecord();
     // Добавление в отчёт
}
```

Главным достоинством линейного поиска является его универсальность и простота. Однако его сложность составляет O(n), что делает его неэффективным при работе с большими объёмами данных.

3.2 Бинарный поиск по стоимости ремонта

Бинарный поиск — это более эффективный алгоритм, который применяется только к предварительно отсортированным данным. В работе он реализован в методе `binarySearchByRepairCost(int repairCost)` и используется для поиска записей с заданной стоимостью ремонта. Предварительная сортировка выполняется алгоритмом вставками по полю `repairCost`, что гарантирует корректность поиска.

Алгоритм работает по классической схеме: сначала вычисляется индекс среднего элемента текущего диапазона. Значение элемента в этой позиции сравнивается с искомым значением. Если оно больше ключа, поиск продолжается в левой части диапазона, исключая правую половину. Если значение меньше, работа продолжается в правой части. Таким образом, с каждым шагом объём данных, в котором производится поиск, уменьшается в два раза, что значительно повышает эффективность по сравнению с линейным методом. Такой подход позволяет существенно сократить количество операций и обеспечивает высокую скорость обработки даже при больших массивах данных.

Пример ключевой логики:

```
if (tempRecord.repairCost > repairCost) {
     right = mid - 1;
} else if (tempRecord.repairCost < repairCost) {
     left = mid + 1;
} else {
     // найдено совпадение
}
```

Преимущество бинарного поиска — логарифмическая сложность `O(log n)`, благодаря чему он эффективен даже при больших объёмах данных. Однако необходимость предварительной сортировки увеличивает общее время выполнения и может быть критична в случае динамически изменяющихся данных.

3.3 Поиск по признаку (марка и максимальная стоимость ремонта)

Реализация выполнена в методе `searchByBrandAndMaxCost(const char* brand, int maxCost)`, где сначала перебираются все записи в хранилище, и каждая проверяется на соответствие сразу двум критериям: совпадение значения поля `brand` с введённой пользователем строкой, а также соответствие поля `repairCost` допустимой стоимости. Если оба условия выполняются, запись добавляется во временный список подходящих результатов.

После отбора подходящих записей они сортируются по дате ремонта в порядке убывания с использованием алгоритма быстрой сортировки. Это позволяет отобразить наиболее актуальные записи в начале списка, что удобно при анализе последних ремонтов, проведённых по заданной марке автомобиля и в пределах заданного бюджета. Найденные записи выводятся на экран в формате, аналогичном другим операциям вывода, и одновременно добавляются в логовый файл с помощью метода `saveReportToFile()`. В файл заносится как описание выполненной операции, так и текстовое представление каждой найденной записи. Такой подход обеспечивает не только визуализацию результата, но и его сохранение для последующего анализа.

Алгоритм сохраняет простоту реализации и обладает линейной сложностью относительно общего количества записей, так как требуется их полная проверка. Однако благодаря последующей сортировке и логированию поиск по признаку демонстрирует высокую практическую ценность, особенно при работе с реальными сценариями анализа ремонтной статистики.

Пример логики:

```
if (strcmp(records[i].brand, brand) == 0 &&
records[i].repairCost <= maxCost) { // найдено совпадение }
```

3.4 Итоги по алгоритмам поиска

В рамках курсовой работе были реализованы три основных метода поиска: линейный, бинарный и поиск по признаку. Каждый из них используется в зависимости от типа данных и поставленной задачи.

Линейный поиск удобен своей универсальностью — он может применяться к неотсортированным данным и не требует предварительной подготовки массива. Это делает его полезным для простых запросов и небольших объёмов информации.

Бинарный поиск, напротив, обеспечивает высокую производительность при больших массивах данных, но требует обязательной предварительной сортировки по ключевому полю. Благодаря логарифмической сложности, он значительно ускоряет выполнение поиска при большом количестве записей.

4 ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

В данной главе подробно рассматриваются все функции, реализованные в рамках курсовой работы. Все они классифицированы по своему назначению и роли в архитектуре системы. Такой подход позволяет структурировать код, облегчить понимание логики работы программы и повысить её поддерживаемость.

Функции условно разделены на несколько групп: интерфейсные (взаимодействие с пользователем, меню), вспомогательные (валидация ввода, выбор параметров), функции работы с данными (чтение, добавление, удаление и редактирование записей), алгоритмы обработки данных (поиск и сортировка), работа с файлами (загрузка, сохранение и логирование), а также отладочные механизмы, применяемые при тестировании.

Такое разделение способствует лучшей модульности кода, ускоряет процесс локализации ошибок и добавления новых функций, а также делает работу более понятной для внешнего анализа и оценки.

4.1 Функции пользовательского интерфейса и навигации

– `start()` — основной метод запуска программы. Инициализирует переменные, запускает бесконечный цикл, в рамках которого на каждом шаге вызывается функция `startMenu()` для отображения главного меню. В зависимости от выбора пользователя вызывается соответствующая функция: добавление, удаление, сортировка, поиск и другие действия. Также реализована очистка экрана и ожидание нажатия клавиши для продолжения работы после выполнения каждой операции;

– `startMenu()` — выводит главное меню с перечнем доступных функций программы (просмотр, добавление, редактирование, поиск, сортировка, статистика и выход). Осуществляет считывание выбора пользователя с клавиатуры, проверяет корректность ввода и возвращает целочисленный код выбранного действия. Используется как основной контроллер навигации внутри метода `start()`;

– `editMenu()` — вспомогательная функция, отображающая подменю редактирования записей. Предоставляет пользователю выбор между изменением данных существующей записи и её удалением. Возвращает соответствующий код действия, который далее обрабатывается функцией `modifyRepairRecord()`.

4.2 Функции работы с записями

– `addRepairRecord()` — отвечает за добавление новой записи в систему. В начале работы вызывается интерфейс ввода, в рамках которого пользователю последовательно предлагается ввести основные характеристики автомобиля: марку, модель, тип проведённого ремонта, стоимость

обслуживания, пробег на момент ремонта, а также точную дату выполнения работ. Каждый параметр проходит обязательную проверку корректности с использованием функций валидации, что позволяет избежать ошибок и неконсистентных данных.

После успешного ввода всех данных создаётся объект структуры `RepairRecord`, в который помещаются введённые значения. Данный объект сохраняется в бинарный файл путём открытия файла в режиме добавления `("ab")` и записи структуры через `fwrite()`. Такой подход обеспечивает последовательное накопление записей без необходимости полной перезаписи файла.

Помимо сохранения в основной файл данных, одновременно формируется текстовая запись, содержащая информацию о добавленном ремонте. Она сохраняется в отдельный отчётный файл, что позволяет вести журнал действий пользователя и использовать его как вспомогательный механизм логирования при анализе работы системы;

– `readRepairRecords()` — реализует чтение и отображение всех существующих записей. Открывает файл в режиме чтения и поочерёдно считывает объекты `RepairRecord`, выводя их на экран с помощью метода `printRepairRecord()`. Первая запись форматируется с заголовками, последующие выводятся списком. После завершения операции формируется и сохраняется текстовый лог с пометкой о просмотре;

– `editRepairRecord()` — функция редактирования записи. При её вызове активируется универсальный метод `modifyRepairRecord(false)`, в котором реализован пошаговый механизм выбора и изменения данных: сначала пользователю отображается список всех существующих записей с индексами, после чего предлагается ввести модель автомобиля, после чего программа находит все подходящие записи и предлагает изменить каждую из них по очереди, далее открывается диалог ввода, где пользователь поочерёдно выбирает поля для редактирования и вводит новые значения, все данные проходят проверку на корректность, после чего обновлённая структура записывается в буфер, а затем необходимый участок бинарного файла перезаписывается на основе вычисленного смещения, что позволяет обновить только нужную запись, не затрагивая остальной файл. Дополнительно создаётся краткий отчёт, который сохраняется в лог-файл и может быть использован для отслеживания пользовательских изменений или проверки корректности функционирования системы;

– `deleteRepairRecord()` — аналогично функции редактирования вызывает метод `modifyRepairRecord(true)`, при этом выбранные пользователем записи удаляются. Реализация основана на копировании всех записей в новый массив за исключением удаляемых, после чего массив записывается в файл заново. Создаётся отчёт о выполнении действия;

– `fieldsEdit(RepairRecord& record)` — вспомогательная функция, вызываемая при редактировании записи. Отображает подменю, позволяющее выбрать конкретное поле для изменения. После выбора пользователю

предлагается ввести новое значение, которое валидируется и заменяет старое. Функция позволяет гибко управлять отдельными атрибутами записи без необходимости полного переопределения объекта.

4.3 Алгоритмы поиска и сортировки

– `linearSearchByModel(const char* model)` — выполняет линейный поиск по модели автомобиля, выводит результаты и сохраняет их в отчёт;

– `binarySearchByRepairCost(int repairCost)` — осуществляет бинарный поиск по стоимости ремонта. Предварительно вызывает сортировку вставками;

– `searchByBrandAndMaxCost(const char* brand, int maxCost)` — фильтрует записи по двум условиям и сортирует результат по убыванию даты ремонта;

– `quickSortByRepairDate()` — загружает данные из файла, сортирует их по дате с использованием быстрой сортировки, сохраняет отсортированные данные обратно;

– `selectionSortByMileage()` — загружает данные из файла, сортирует записи по пробегу выбором, сохраняет отсортированные данные обратно;

– `insertionSortByRepairCost()` — загружает данные из файла, сортирует записи по стоимости ремонта вставками, сохраняет отсортированные данные обратно.

4.4 Вспомогательные функции для работы с файлами

– `loadFromFile()` — функция, обеспечивающая загрузку всех записей из бинарного файла данных в динамический массив объектов `RepairRecord`. Она открывает файл в режиме чтения `(rb)`, выделяет необходимую память в куче для хранения всех записей, затем последовательно считывает каждую запись с помощью `fread()` и помещает в массив. Возвращает указатель на созданный массив или `nullptr`, если чтение не удалось. Также учитывается количество записей, рассчитанное по размеру файла и размеру структуры;

– `saveToFile(RepairRecord* list)` — принимает на вход указатель на массив структур `RepairRecord`, предварительно отсортированный или отредактированный пользователем. Открывает файл в режиме записи `(wb)`, полностью очищая его содержимое, и записывает все объекты из массива в файл с помощью `fwrite()`. Данная функция гарантирует, что актуальные данные сохраняются без остаточных значений от предыдущих операций. Используется при применении сортировок и массовом редактировании записей;

– `saveReportToFile(const char* content)` — отвечает за логирование всех действий пользователя, таких как поиск, добавление, удаление, сортировка и редактирование записей. При каждом вызове

соответствующей функции формируется текстовая строка, отражающая суть действия и, при необходимости, содержащая ключевые параметры (например, модель автомобиля, дата или стоимость ремонта). Данная строка передаётся в функцию как аргумент `content`. Внутри функции файл отчета (например, `report.txt`) открывается в режиме добавления `("a")`, что позволяет не перезаписывать содержимое, а дописывать новую информацию в конец. Каждая операция сопровождается добавлением временной метки с точным временем выполнения, благодаря чему формируется последовательный и достоверный журнал действий. Этот журнал может быть использован для анализа пользовательской активности, отладки, обратной связи на защите работы, а также при выявлении ошибок или некорректных изменений данных в процессе эксплуатации программы.

4.5 Функции валидации и ввода

– `getValidatedString(...)` — функция, обеспечивающая безопасный и корректный ввод строковых данных от пользователя. Принимает на вход сообщение-подсказку `(prompt)`, буфер для ввода `(char* buffer)`, тип символов (цифры, буквы, пробелы и пр.), минимальную и максимальную длину строки, а также сообщение об ошибке. После запроса ввода строка проверяется функцией `validateStr(...)`. Если строка не соответствует заданным критериям (например, содержит недопустимые символы или имеет некорректную длину), пользователю выводится сообщение об ошибке, и ввод повторяется. Это обеспечивает устойчивость к ошибочным действиям пользователя и предотвращает некорректные данные;

– `validateStr(...)` — вспомогательная функция, вызываемая внутри `getValidatedString`. Проверяет, соответствует ли введённая строка условиям: допустимые символы, диапазон длины и тип ввода (например, только буквы или буквы с цифрами). Возвращает логическое значение (`true` или `false`), определяя, можно ли принять строку как корректную. Именно на основе этой функции строится повторный цикл запроса данных при неправильном вводе;

– `getValidatedInt(...)` — функция, аналогичная `getValidatedString`, но предназначенная для ввода целых чисел. Принимает подсказку, переменную для хранения значения, нижнюю и верхнюю границы, а также сообщение об ошибке. Если пользователь вводит нечисловой символ или число, выходящее за пределы допустимого диапазона, функция очищает поток ввода, выводит сообщение и запрашивает ввод заново. Это повышает надёжность обработки пользовательского ввода и защищает программу от краха при некорректных данных;

– `validateInt(...)` — внутренняя логика, проверяющая введённое число на соответствие заданным границам. Получает ссылку на переменную и границы диапазона, а также сообщение об ошибке. Если значение выходит за допустимые пределы, функция возвращает `false`, инициируя повторный ввод

в `getValidatedInt`. Обеспечивает строгую проверку целочисленных значений;

– `selectRepairType()` — функция, отображающая пользователю перечень возможных типов ремонта: техническое обслуживание, кузовной ремонт, замена деталей, ремонт двигателя, электрики, тормозной системы и прочее. Пользователь выбирает нужный пункт с помощью ввода соответствующего номера. Ввод обрабатывается и проверяется на допустимость, после чего возвращается соответствующее значение из перечисления `RepairType`. Если введено недопустимое значение, автоматически возвращается значение Other, что исключает сбои и делает обработку ввода более надёжной. Это позволяет стандартизировать значения типа ремонта в системе, исключая ручной текстовый ввод и тем самым предотвращая ошибки.

4.6 Отладочные функции

`randomFill(int amount)` — вспомогательная отладочная функция, предназначенная для автоматического заполнения файла случайными тестовыми записями. Функцию можно вызвать только при наличии директивы препроцессора `#define DEBUG`, что исключает её использование в финальной версии программы. Это позволяет разработчику протестировать функциональность системы на основе искусственно сгенерированных данных, не вводя их вручную.

Внутри функции происходит генерация объектов типа `RepairRecord` с псевдослучайными значениями для всех полей:

– Марка и модель автомобиля выбираются из заранее заданных массивов строк (например, "Toyota", "Ford", "BMW" и т. д.);

– Тип ремонта (`repairType`) выбирается случайным образом из доступных значений перечисления `RepairType`;

– Стоимость (`repairCost`) и пробег (`mileage`) задаются как случайные числа в заданных диапазонах (например, от 100 до 10000 и от 1000 до 300000 соответственно);

– Дата ремонта (`repairDate`) также формируется случайно, с учётом допустимых границ (день от 1 до 31, месяц от 1 до 12, год в пределах реального срока эксплуатации).

Каждая сгенерированная запись записывается в бинарный файл через `fwrite()`. В функции используется цикл по количеству `amount`, указанному при вызове функции, что позволяет гибко варьировать объём тестовых данных.

Функция существенно ускоряет разработку, позволяя многократно запускать сортировки, поиски и анализ статистики на большом объёме записей без участия пользователя. Это делает её неотъемлемой частью отладки, особенно при тестировании предельных сценариев и нагрузок.

4.7 Итог реализации пользовательских функций

Подводя итог реализации пользовательских функций, следует подчеркнуть, что они охватывают весь цикл обработки информации — от первичного ввода и проверки данных до логирования, сортировки, поиска и отображения результатов. Каждая функция в работе не только решает отдельную задачу, но и тесно связана с другими элементами системы, образуя целостную архитектуру, где отсутствуют изолированные или неиспользуемые участки кода. Такое единство повышает эффективность разработки и надёжность эксплуатации.

Функции управления записями реализованы с учётом базовой проверки ввода. Это помогает избежать ошибок при работе с бинарными файлами. Поиск и сортировка применяются по ситуации. Логирование даёт возможность отслеживать действия пользователя.

Важно отметить, что структура функций в работе была построена с ориентацией на модульность и расширяемость: каждый компонент можно адаптировать под изменившиеся требования без необходимости кардинальной переработки остальных частей. Это особенно ценно в реальных условиях, где спецификация может измениться на поздних этапах разработки. Такой подход также упрощает тестирование и внесение правок — благодаря чёткой логической границе между функциональными блоками.

Итогом становится не просто набор функций, решающих локальные задачи, а продуманная и устойчивая система, в которой все элементы логически выверены и направлены на достижение общей цели — эффективного управления и анализа информации о ремонтах автомобилей. Именно такая организация функций демонстрирует зрелость проектного подхода и соответствует требованиям к современным учебным и производственным системам.

5 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

Данная глава посвящена описанию интерфейса и логики взаимодействия пользователя с программой. Основная цель — продемонстрировать пошаговую работу системы: от запуска до выполнения операций и формирования лога. Программа ориентирована на простоту использования и устойчивость к ошибочным действиям, обеспечивая интуитивно понятное взаимодействие.

После запуска программы вызывается метод start(), внутри которого отображается главное меню. Пользователь может выбрать нужное действие — добавление, удаление, редактирование, сортировка, поиск, просмотр записей или выход из программы.

```
 1. View all records
 2. Add a new record
 3. Edit a record
 4. Delete a record
 5. Find a record by car model (Linear search)
 6. Find a record by repair cost (Binary search)
 7. Find records by brand and max cost
 8. Sort records by date (Quick sort)
 9. Sort records by mileage (Selection sort)
10. Sort records by repair cost (Insertion sort)
11. Repair statistics*
 Any key to exit

* For each repair type, display a list of works
sorted in ascending order by date.
Find the most expensive and the cheapest repair.

Please make a choice: 
```

Рисунок 5.1 – Главное меню при запуске программы

После выбора действия вызывается соответствующая функция. Каждая операция сопровождается подробными инструкциями и встроенной валидацией данных.

При выполнении сортировки — по дате, стоимости или пробегу — программа сначала обрабатывает данные и затем выводит сообщение об успешной операции.

```
Please make a choice: 8
File was sorted successfully.
Press any key to continue..
```

Рисунок 5.2 – Сообщение об успешной сортировке

При добавлении записи программа отображает форму с полями для ввода — марка, модель, тип ремонта, стоимость, пробег, дата. После ввода каждого значения осуществляется проверка на корректность. Если данные верны, запись сохраняется в файл. Этот процесс показан на скриншоте ниже.

```
Please make a choice: 2
Enter car brand: BMW
Enter car model: Msd/.
Car model must have length between 2 and 24,
contain numbers from 0 to 9,
letters from a to z and A to Z and space.
Try again: M5 F90
Choose repair type:
 1. Maintenance
 2. Body Repair
 3. Parts Replacement
 4. Engine Repair
 5. Electrical Repair
 6. Brake System Repair
 Any key to select Other
Please make choice: 4
Enter repair cost in dollars: -100
Repair cost must be a number between 1 and 1000000.
Try again: 12400
Enter mileage in kilometers: 1300
Enter date of repair:
Day: 11
Month: 10
Year: 2025
Repair record added successfully.
Press any key to continue..
```

Рисунок 5.3 – Пример ввода новой записи

В случае редактирования или удаления программа предлагает пользователю ввести модель автомобиля. Затем последовательно отображаются все записи, соответствующие указанной модели, и предлагается изменить или удалить каждую из них. Такой подход позволяет гибко управлять данными.

```
Enter car model to edit: M5 F90
Found record:
-----------------------------------------
Car brand: BMW
Car model: M5 F90
Repair type: Engine Repair
Repair cost: $12300
Mileage: 2400km
Date: 11.10.2025
-----------------------------------------
What do you want to edit?
 1. Car brand
 2. Car model
 3. Repair type
 4. Repair cost
 5. Mileage
 6. Repair date
 Any key to nothing
Please make choice:
```

Рисунок 5.4 – Интерфейс редактирования найденных записей

Поиск осуществляется по ключевым полям — модели или стоимости ремонта. После ввода искомого значения отображается список подходящих записей в отформатированном виде. Если совпадений не найдено, программа сообщает об этом.

```
Please make a choice: 7
Enter car brand: BMW
Enter max repair cost in dollars: 12500
Found records:
-----------------------------------------
Car brand: BMW
Car model: M5 F90
Repair type: Engine Repair
Repair cost: $12300
Mileage: 2400km
Date: 11.10.2025
-----------------------------------------
```

Рисунок 5.5 – Результат успешного поиска и форматированный вывод

Кроме того, при каждой операции в фоновом режиме формируется и обновляется текстовый отчет (лог), содержащий сведения о действиях пользователя и результатах операций. Этот отчет создается с использованием функции `saveReportToFile`, которая принимает строку текста и сохраняет её в конец файла отчета, как правило, `report.txt`. Лог формируется динамически и отражает последовательность действий пользователя: добавление, удаление, редактирование, сортировка, выполнение поиска и просмотр записей.

```
═══ Report added on: Thu May 22 23:43:41 2025
File sorted by quick sort by date

═══ Report added on: Thu May 22 23:44:23 2025
User has viewed all repair records.

═══ Report added on: Thu May 22 23:48:51 2025
Added record :
BMW M5 F90, Engine Repair, $12400, 1300km, 11.10.2025
```

Рисунок 5.6 – Пример содержимого отчётного файла

Таким образом, лог содержит как действия пользователя, так и конкретные данные, с которыми эти действия были связаны. Файл отчета открывается в режиме добавления, чтобы не перезаписывать предыдущую информацию. Такой подход позволяет создать полный и непрерывный хронологический журнал использования программы, который может быть

использован для анализа поведения пользователя, отладки и проверки корректности выполнения операций.

Для обработки некорректных ситуаций предусмотрены механизмы повторного запроса ввода с пояснением причин ошибки. Например, при вводе неверной стоимости пользователю предлагается повторить попытку.

```
Enter repair cost in dollars: -100
Repair cost must be a number between 1 and 1000000.
Try again: 12400
```

Рисунок 5.7 – Обработка ошибки при вводе стоимости

Таким образом, программа предоставляет последовательный и понятный пользовательский опыт. Благодаря пошаговому управлению, проверкам, логированию и визуальной структуре меню, система остаётся доступной для пользователя с любым уровнем подготовки и устойчивой к ошибкам при эксплуатации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были достигнуты все поставленные цели и решены задачи, обозначенные во введении. Разработано программное средство, позволяющее вести учёт ремонтных записей автомобилей с возможностью сортировки по дате, пробегу и стоимости, линейного и бинарного поиска, а также поиска по признаку по марке и максимальной стоимости ремонта. Система обеспечивает отображение статистики по типам ремонта, определение самой дорогой и дешёвой записи, а также предоставляет функции добавления, редактирования, удаления и просмотра информации. Все операции сопровождаются автоматическим формированием текстового отчёта.

Построенная архитектура отличается модульностью, устойчивостью и возможностью дальнейшего расширения: добавления графического интерфейса, поддержки других форматов хранения или сетевого взаимодействия. Программа может быть использована как в учебных целях, так и в качестве основы для практического применения в небольших сервисных структурах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Седжвик Р. Фундаментальные алгоритмы на C++. Анализ / Структуры данных / Сортировка / Поиск / Р. Седжвик. — Москва: «ДиаСофт», 2001. — 688 с.

[2] Программирование: принципы и практика с использованием C++, 2-е изд. / Пер. с англ. — М.: ООО «И.Д. Вильяме», 2016. — 1328 с.: ил. — Парал. тит. англ.

[3] Лафоре Р. Объектно-ориентированное программирование в C++. Классика Computer Science. 4 изд. — СПб.: Питер, 2018. — 928 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

```
#include <conio.h>
#include <cstdio>
#include <iostream>
#define DEBUG
using namespace std;

class CarRepairSystem {
public:
     CarRepairSystem(const char* recordsPath);
     ~CarRepairSystem();

     void start();
#ifdef DEBUG
     void randomFill(int amount);
#endif

private:
     enum class RepairType {
          Maintenance,          // ТО
          BodyRepair,           // кузовной ремонт
          PartsReplacement,     // замена деталей
          EngineRepair,         // ремонт двигателя
          ElectricalRepair,     // ремонт электрики
          BrakeSystemRepair,    // ремонт тормозной системы
          Other                 // другое
     };

     static const int AMOUNT_OF_REPAIR_TYPES;
     static const char* repairTypeStrings[];

     class RepairDate {
     public:
          int day;
          int month;
          int year;
          bool operator > (RepairDate other);
          bool operator < (RepairDate other);
     };

     class RepairRecord {
     public:
          char brand[25]{};
          char model[25]{};
          RepairType repairType;
          int repairCost;
          int mileage;
          RepairDate repairDate;
```

Продолжение приложения А

```
            RepairRecord(const char* brand = "", const char*
        model = "", RepairType repairType = {}, int repairCost
        = 0, int mileage = 0, RepairDate repairDate = {});
            void printRepairRecord(bool isFirst = false)
const;
            void toString(char* buffer);
        };

        char recordsPath[255]{};
        int recordsCount;


        void readRepairRecords();
        void addRepairRecord();
        void editRepairRecord();
        void deleteRepairRecord();
        void modifyRepairRecord(bool isDelete);

        void linearSearchByModel(const char* model);
        void binarySearchByRepairCost(int repairCost);
        void searchByBrandAndMaxCost(const char* brand, int
maxCost);

        void quickSortByRepairDate();
        void selectionSortByMileage();
        void insertionSortByRepairCost();

        void repairStats();
        void displayRepairsByType();
        void findMostExpensiveAndCheapestRepair();

        int startMenu();
        int editMenu();
        void fieldsEdit(RepairRecord& record);
        RepairType selectRepairType();

        RepairRecord* loadFromFile() const;
        void saveToFile(RepairRecord* list) const;
        void saveReportToFile(const char* content);

        void quickSort(RepairRecord* list, int left, int
right);
        void selectionSort(RepairRecord* list);
        void insertionSort(RepairRecord* list);

        bool validateStr(char* buffer, int type, int minLen,
int maxLen, const char* errorMsg);
        void getValidatedString(const char* prompt, char*
buffer, int type, int minLen, int maxLen, const char* errorMsg);
        bool validateInt(int& value, int min, int max, const
char* errorMsg);
```

Продолжение приложения А

```
          void getValidatedInt(const char* prompt, int& value,
     int min, int max, const char* errorMsg);
     };

     const int CarRepairSystem::AMOUNT_OF_REPAIR_TYPES = 7;

     const char*
CarRepairSystem::repairTypeStrings[CarRepairSystem::AMOUNT_OF_RE
PAIR_TYPES] = {
               "Maintenance", "Body Repair", "Parts Replacement",
"Engine Repair",
               "Electrical Repair", "Brake System Repair",
"Other"
     };

     CarRepairSystem::CarRepairSystem(const char* recordsPath) {
          strcpy_s(this->recordsPath, sizeof(this->recordsPath),
recordsPath);

          FILE* recordsFile = nullptr;
          if (fopen_s(&recordsFile, recordsPath, "rb") ||
recordsFile == nullptr) {
               recordsCount = 0;
               return;
          }

          fseek(recordsFile, 0, SEEK_END);
          long recordsFileSize = ftell(recordsFile);
          fclose(recordsFile);

          if (recordsFileSize < 0) { recordsCount = 0; }
          else { recordsCount = recordsFileSize /
sizeof(RepairRecord); }
     }

     CarRepairSystem::~CarRepairSystem() {

     }

     void CarRepairSystem::start() {
          char tempStr[25]{};
          int tempInt;
          while (true) {
               tempInt = 0;
               for (int i = 0; tempStr[i]; i++) { tempStr[i] =
'\0'; }
               switch (startMenu()) {
               case 1: readRepairRecords(); break;
               case 2: addRepairRecord(); break;
               case 3: editRepairRecord(); break;
               case 4: deleteRepairRecord(); break;
```

Продолжение приложения А

```
                case 5:
                     getValidatedString("Enter car model: ",
tempStr, 1, 2, 24, "Car model must have length between 2 and 24,
contain numbers from 0 to 9, letters from a to z and A to Z and
space.\nTry again: ");
                     linearSearchByModel(tempStr);
                     break;
                case 6:
                     getValidatedInt("Enter repair cost in
dollars: ", tempInt, 1, 1000000, "Repair cost must be a number
between 1 and 1000000.\nTry again: ");
                     binarySearchByRepairCost(tempInt);
                     break;
                case 7:
                     getValidatedString("Enter car brand: ",
tempStr, 0, 3, 24, "Car brand must have length between 3 and 24,
contain letters from a to z and A to Z.\nTry again: ");
                     getValidatedInt("Enter max repair cost in
dollars: ", tempInt, 1, 1000000, "Repair cost must be a number
between 1 and 1000000.\nTry again: ");
                     searchByBrandAndMaxCost(tempStr, tempInt);
                     break;
                case 8: quickSortByRepairDate(); break;
                case 9: selectionSortByMileage(); break;
                case 10: insertionSortByRepairCost(); break;
                case 11: repairStats(); break;
                default:
                     return;
                }
                cout << "Press any key to continue..";
                _getch();
                system("cls");
           }
      }

      void CarRepairSystem::addRepairRecord() {
           FILE* recordsFile;
           if (fopen_s(&recordsFile, recordsPath, "ab")) {
                cout << "Error opening records file." << endl;
                return;
           }
           char tempBrand[25]{};
           char tempModel[25]{};
           RepairType tempRepairType;
           int tempRepairCost;
           int tempMileage;
           RepairDate tempRepairDate = {};

           getValidatedString("Enter car brand: ", tempBrand, 0,
3, 24, "Car brand must have length between 3 and 24, contain
letters from a to z and A to Z.\nTry again: ");
```

Продолжение приложения А

```
        getValidatedString("Enter car model: ", tempModel, 1,
    2, 24, "Car model must have length between 2 and 24, contain
    numbers from 0 to 9, letters from a to z and A to Z and
    space.\nTry again: ");
        tempRepairType = selectRepairType();
        getValidatedInt("Enter repair cost in dollars: ",
tempRepairCost, 1, 1000000, "Repair cost must be a number
between 1 and 1000000.\nTry again: ");
        getValidatedInt("Enter mileage in kilometers: ",
tempMileage, 1, 5000000, "Mileage must be a number between 1 and
5000000.\nTry again: ");
        cout << "Enter date of repair:\n";
        getValidatedInt("Day: ", tempRepairDate.day, 1, 31,
"Day must be a number between 1 and 31.\nTry again: ");
        getValidatedInt("Month: ", tempRepairDate.month, 1, 12,
"Month must be a number between 1 and 12.\nTry again: ");
        getValidatedInt("Year: ", tempRepairDate.year, 1900,
2100, "Year must be a number between 1900 and 2100.\nTry again:
");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cin.clear();

        RepairRecord tempRecord(tempBrand, tempModel,
tempRepairType, tempRepairCost, tempMileage, tempRepairDate);

        char reportContent[512]{};
        strcat_s(reportContent, sizeof(reportContent), "Added
record :\n");
        char recordStr[256]{};
        tempRecord.toString(recordStr);
        strcat_s(reportContent, sizeof(reportContent),
recordStr);
        saveReportToFile(reportContent);

        fwrite(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
        cout << "Repair record added successfully." << endl;
        recordsCount++;
        fclose(recordsFile);
    }

    void CarRepairSystem::readRepairRecords() {
        FILE* recordsFile;
        if (fopen_s(&recordsFile, recordsPath, "rb")) {
            cout << "Error opening records file." << endl;
            return;
        }

        RepairRecord tempRecord;
        fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
```

Продолжение приложения А

```
          tempRecord.printRepairRecord(true);
          while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) { tempRecord.printRepairRecord(); }
          saveReportToFile("User has viewed all repair
records.\n");

          fclose(recordsFile);
     }

     void CarRepairSystem::editRepairRecord() {
          modifyRepairRecord(false);
     }

     void CarRepairSystem::deleteRepairRecord() {
          modifyRepairRecord(true);
     }

     void CarRepairSystem::linearSearchByModel(const char* model)
{
          FILE* recordsFile;

          if (fopen_s(&recordsFile, recordsPath, "rb")) {
               cout << "Error opening records file." << endl;
               return;
          }

          RepairRecord tempRecord;
          bool isFound = false;
          bool isFirst = true;
          char reportContent[10000]{};
          strcat_s(reportContent, sizeof(reportContent), "Linear
search by model ");
          strcat_s(reportContent, sizeof(reportContent), model);
          strcat_s(reportContent, sizeof(reportContent), ":\n");

          while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
               if (strcmp(tempRecord.model, model) == 0) {
                    if (isFirst) cout << "Found records: " <<
endl;
                    tempRecord.printRepairRecord(isFirst);

                    char recordStr[256];
                    tempRecord.toString(recordStr);
                    strcat_s(reportContent,
sizeof(reportContent), recordStr);

                    isFound = true;
                    isFirst = false;
               }
          }
```

Продолжение приложения А

```
          if (!isFound) {
               cout << "Records not found." << endl;
               strcat_s(reportContent, sizeof(reportContent),
"Records not found.\n");
          }
          saveReportToFile(reportContent);
          fclose(recordsFile);
     }

     void CarRepairSystem::binarySearchByRepairCost(int
repairCost) {
          insertionSortByRepairCost();
          cout << "File was sorted by repair cost for binary
search." << endl;

          FILE* recordsFile;

          if (fopen_s(&recordsFile, recordsPath, "rb")) {
               cout << "Error opening records file." << endl;
               return;
          }

          int left = 0;
          int right = recordsCount - 1;
          int mid;
          RepairRecord tempRecord;
          bool isFound = false;

          while (left <= right) {
               mid = left + (right - left) / 2;
               fseek(recordsFile, mid * sizeof(RepairRecord),
SEEK_SET);
               fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);

               if (tempRecord.repairCost > repairCost) { right =
mid - 1; }
               else if (tempRecord.repairCost < repairCost) {
left = mid + 1; }
               else {
                    isFound = true;
                    break;
               }
          }
          if (!isFound) {
               cout << "Records not found." << endl;
               fclose(recordsFile);
               saveReportToFile("Binary search with repair
cost:\nRecords not found.\n");
               return;
          }
```

Продолжение приложения А

```
          int first = mid;
          int last = mid;

          while (first > 0) {
               fseek(recordsFile, (first - 1) *
sizeof(RepairRecord), SEEK_SET);
               fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
               if (tempRecord.repairCost == repairCost) { first--
; }
               else { break; }
          }
          while (last < recordsCount - 1) {
               fseek(recordsFile, (last + 1) *
sizeof(RepairRecord), SEEK_SET);
               fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
               if (tempRecord.repairCost == repairCost) { last++;
}
               else { break; }
          }
          char reportContent[10000] = "Binary search with repair
cost $";
          char tempBuffer[32]{};
          sprintf_s(tempBuffer, sizeof(tempBuffer), "%d",
repairCost);
          strcat_s(reportContent, sizeof(reportContent),
tempBuffer);
          strcat_s(reportContent, sizeof(reportContent), ":\n");

          cout << "Found records with repair cost $" <<
repairCost << ":" << endl;
          for (int i = first; i <= last; i++) {
               fseek(recordsFile, i * sizeof(RepairRecord),
SEEK_SET);
               fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
               tempRecord.printRepairRecord(i == first);
               char recordStr[256];
               tempRecord.toString(recordStr);
               strcat_s(reportContent, sizeof(reportContent),
recordStr);
          }
          fclose(recordsFile);
          saveReportToFile(reportContent);
     }

     void CarRepairSystem::searchByBrandAndMaxCost(const char*
brand, int maxCost) {
          FILE* recordsFile;
```

Продолжение приложения А

```
          if (fopen_s(&recordsFile, recordsPath, "rb")) {
               cout << "Error opening records file." << endl;
               return;
          }

          RepairRecord tempRecord;
          bool isFound = false;
          int resultCounter = 0;

          while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
               if (strcmp(tempRecord.brand, brand) == 0 &&
tempRecord.repairCost <= maxCost) {
                    resultCounter++;
                    isFound = true;
               }
          }
          if (!isFound) {
               cout << "Records not found." << endl;
               fclose(recordsFile);
               return;
          }

          RepairRecord* list = new RepairRecord[resultCounter];
          int position = 0;
          rewind(recordsFile);
          while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
               if (strcmp(tempRecord.brand, brand) == 0 &&
tempRecord.repairCost <= maxCost) {
                    list[position] = tempRecord;
                    position++;
               }
          }

          fclose(recordsFile);

          quickSort(list, 0, resultCounter - 1);
          cout << "Found records:" << endl;
          char reportContent[10000] = "Search by brand and max
cost: ";
          char tempBuffer[32]{};
          strcat_s(reportContent, sizeof(reportContent), brand);
          sprintf_s(tempBuffer, sizeof(tempBuffer), ", %d",
maxCost);
          strcat_s(reportContent, sizeof(reportContent),
tempBuffer);
          strcat_s(reportContent, sizeof(reportContent), ":\n");
          for (int i = resultCounter; i > 0; i--) {
               list[i - 1].printRepairRecord(i == resultCounter);
               char recordStr[256];
```

Продолжение приложения А

```
                list[i - 1].toString(recordStr);
                strcat_s(reportContent, sizeof(reportContent),
recordStr);
            }
            saveReportToFile(reportContent);
            delete[] list;
            list = nullptr;
        }

        void CarRepairSystem::quickSortByRepairDate() {
            RepairRecord* list = loadFromFile();
            if (list == nullptr) {
                cout << "Error loading data from file." << endl;
                return;
            }
            quickSort(list, 0, recordsCount - 1);
            saveToFile(list);
            saveReportToFile("File sorted by quick sort by
date\n");
            cout << "File was sorted successfully." << endl;
        }

        void CarRepairSystem::selectionSortByMileage() {
            RepairRecord* list = loadFromFile();
            if (list == nullptr) {
                cout << "Error loading data from file." << endl;
                return;
            }
            selectionSort(list);
            saveToFile(list);
            saveReportToFile("File sorted by selection sort by
mileage\n");
            cout << "File was sorted successfully." << endl;
        }

        void CarRepairSystem::insertionSortByRepairCost() {
            RepairRecord* list = loadFromFile();
            if (list == nullptr) {
                cout << "Error loading data from file." << endl;
                return;
            }
            insertionSort(list);
            saveToFile(list);
            saveReportToFile("File sorted by insertion sort by
cost\n");
            cout << "File was sorted successfully." << endl;
        }

        void CarRepairSystem::repairStats() {
            displayRepairsByType();
            findMostExpensiveAndCheapestRepair();
```

Продолжение приложения А

```
     }

     void CarRepairSystem::displayRepairsByType() {
          RepairRecord* list = loadFromFile();
          quickSort(list, 0, recordsCount - 1);
          bool isFound;
          bool isFirst;
          char reportContent[10000]{};
          for (int i = 0; i < AMOUNT_OF_REPAIR_TYPES; i++) {
               isFound = false;
               isFirst = true;
               cout << repairTypeStrings[i] << ": " << endl;
               strcat_s(reportContent, sizeof(reportContent),
repairTypeStrings[i]);
               strcat_s(reportContent, sizeof(reportContent),
":\n");
               for (int j = 0; j < recordsCount; j++) {
                    if (list[j].repairType ==
static_cast<RepairType>(i)) {

                         list[j].printRepairRecord(isFirst);

                         char recordStr[256];
                         list[j].toString(recordStr);
                         strcat_s(reportContent,
sizeof(reportContent), recordStr);

                         isFound = true;
                         isFirst = false;
                    }
               }
               if (!isFound) {
                    cout << "Records not found." << endl;
                    strcat_s(reportContent,
sizeof(reportContent), "Records not found.\n");
               }
          }
          saveReportToFile(reportContent);
          delete[] list;
          list = nullptr;
     }

     void CarRepairSystem::findMostExpensiveAndCheapestRepair() {
          /*RepairRecord* list = loadFromFile();
          if (list == nullptr) {
               cout << "Error in calculating most expensive and
cheapest repairs. (Error loading data from file)" << endl;
               return;
          }
          insertionSort(list);
```

Продолжение приложения А

```
        cout << "Most expensive repair: $" <<
    list[0].repairCost << endl;
        cout << "The cheapest repair: $" << list[recordsCount -
1].repairCost << endl;
        delete[] list;
        list = nullptr;*/


        FILE* recordsFile;
        if (fopen_s(&recordsFile, recordsPath, "rb")) {
            cout << "Error opening records file." << endl;
            return;
        }
        RepairRecord mostExpensive;
        RepairRecord cheapest;
        RepairRecord tempRecord;
        fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile);
        cheapest = mostExpensive = tempRecord;
        while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
            if (tempRecord.repairCost >
mostExpensive.repairCost) { mostExpensive = tempRecord; }
            if (tempRecord.repairCost < cheapest.repairCost) {
cheapest = tempRecord; }
        }
        cout << "Most expensive repair: $" <<
mostExpensive.repairCost << endl;
        cout << "The cheapest repair: $" << cheapest.repairCost
<< endl;
        char reportContent[10000] = "Most expensive repair: $";
        char tempBuffer[32]{};
        sprintf_s(tempBuffer, sizeof(tempBuffer), "%d",
mostExpensive.repairCost);
        strcat_s(reportContent, sizeof(reportContent),
tempBuffer);
        strcat_s(reportContent, sizeof(reportContent), "\nThe
cheapest repair: $");
        sprintf_s(tempBuffer, sizeof(tempBuffer), "%d",
cheapest.repairCost);
        strcat_s(reportContent, sizeof(reportContent),
tempBuffer);
        strcat_s(reportContent, sizeof(reportContent), "\n");
        saveReportToFile(reportContent);
    }
```

Продолжение приложения А

```
#ifdef DEBUG
void CarRepairSystem::randomFill(int amount) {
	const char cars[5][6][25] = {
		{"BMW", "3 Series", "5 Series", "X5", "M4", "iX"},
		{"Toyota", "Corolla", "Camry", "RAV4", "Supra",
"Land Cruiser"},
		{"Mercedes", "C Class", "E Class", "GLE", "S
Class", "AMG GT"},
		{"Ford", "Mustang", "F150", "Explorer", "Focus",
"Bronco"},
		{"Audi", "A4", "A6", "Q7", "RS5", "etron GT"}
	};
	FILE* recordsFile;
	if (fopen_s(&recordsFile, recordsPath, "ab")) {
		cout << "Error opening records file." << endl;
		return;
	}
	srand(time(NULL));
	RepairDate randDate{};
	int randBrand;

	for (int i = 0; i < amount; i++) {
		randDate.day = rand() % 31 + 1;
		randDate.month = rand() % 12 + 1;
		randDate.year = rand() % 10 + 2015;
		randBrand = rand() % 5;

		RepairRecord randRecord(cars[randBrand][0],
cars[randBrand][rand() % 5 + 1], static_cast<RepairType>(rand()
% AMOUNT_OF_REPAIR_TYPES), rand() % 9901 + 100, rand() % 290001
+ 10000, randDate);
		fwrite(&randRecord, sizeof(RepairRecord), 1,
recordsFile);
		recordsCount++;
	}
	fclose(recordsFile);
}
#endif

void CarRepairSystem::modifyRepairRecord(bool isToDelete) {
	FILE* recordsFile;
	FILE* tempFile;

	if (fopen_s(&recordsFile, recordsPath, "rb")) {
		cout << "Error opening records file." << endl;
		return;
	}

	if (fopen_s(&tempFile, "temp.bin", "wb")) {
		cout << "Error opening temporary file." << endl;
		fclose(recordsFile);
```

Продолжение приложения А

```
                return;
            }

            RepairRecord tempRecord;
            bool isFound = false;
            bool isFirst = true;
            char reportContent[10000]{};
            char tempModel[25]{};
            char prompt[28]{};
            if (isToDelete) { strcpy_s(prompt, sizeof(prompt),
"Enter car model to delete: "); }
            else { strcpy_s(prompt, sizeof(prompt), "Enter car
model to edit: "); }
            getValidatedString(prompt, tempModel, 1, 2, 24, "Car
model must have length between 2 and 24, contain numbers from 0
to 9, letters from a to z and A to Z and space.\nTry again: ");

            while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
                if (strcmp(tempRecord.model, tempModel) == 0) {
                    if (isFirst && isToDelete) {
                        cout << "Deleted records: " << endl;
                        strcat_s(reportContent,
sizeof(reportContent), "Deleted records: \n");
                    }
                    if (!isToDelete) cout << "Found record: " <<
endl;
                    tempRecord.printRepairRecord(isFirst);
                    if (!isToDelete) {
                        fieldsEdit(tempRecord);
                        fwrite(&tempRecord,
sizeof(RepairRecord), 1, tempFile);
                    }
                    else { recordsCount--; }
                    isFound = true;
                    if (isToDelete) {
                        isFirst = false;
                        char recordStr[256];
                        tempRecord.toString(recordStr);
                        strcat_s(reportContent,
sizeof(reportContent), recordStr);
                    }
                }
                else { fwrite(&tempRecord, sizeof(RepairRecord),
1, tempFile); }
            }

            if (isFound) {
                if (isToDelete) { cout << "Records deleted
successfully." << endl; }
```

Продолжение приложения А

```
          else { cout << "Records edited successfully." << endl;
     }
          }
          else {
               cout << "Records not found." << endl;
               if(isToDelete){ strcat_s(reportContent,
sizeof(reportContent), "Records not found(deleting).\n"); }
               else { saveReportToFile("Records not
found(editing).\n"); }
          }
          if (isToDelete) { saveReportToFile(reportContent); }
          fclose(recordsFile);
          fclose(tempFile);

          remove(recordsPath);
          if (rename("temp.bin", recordsPath)) { cout << "Error
renaming file." << endl; }
          if (!isToDelete) {
               cin.ignore(numeric_limits<streamsize>::max(),
'\n');
               cin.clear();
          }
     }

     int CarRepairSystem::startMenu() {
          cout << " 1. View all records" << endl
               << " 2. Add a new record" << endl
               << " 3. Edit a record" << endl
               << " 4. Delete a record" << endl
               << " 5. Find a record by car model (Linear
search)" << endl
               << " 6. Find a record by repair cost (Binary
search)" << endl
               << " 7. Find records by brand and max cost" <<
endl
               << " 8. Sort records by date (Quick sort)" << endl
               << " 9. Sort records by mileage (Selection sort)"
<< endl
               << "10. Sort records by repair cost (Insertion
sort)" << endl
               << "11. Repair statistics*" << endl
               << " Any key to exit" << endl
               << "\n* For each repair type, display a list of
works sorted in ascending order by date. Find the most expensive
and the cheapest repair.\n" << endl
               << "Please make a choice: ";
          int choice;
          cin >> choice;
          cin.ignore(numeric_limits<streamsize>::max(), '\n');
          cin.clear();
          return (cin.fail()) ? 0 : choice;
```

Продолжение приложения А

```
     }

     int CarRepairSystem::editMenu() {
          cout << "What do you want to edit?" << endl
               << " 1. Car brand" << endl
               << " 2. Car model" << endl
               << " 3. Repair type" << endl
               << " 4. Repair cost" << endl
               << " 5. Mileage" << endl
               << " 6. Repair date" << endl
               << " Any key to nothing" << endl
               << "Please make choice: ";
          int choice;
          cin >> choice;
          return (cin.fail()) ? 0 : choice;
     }

     void CarRepairSystem::fieldsEdit(RepairRecord& record) {
          int tempInt;
          char tempStr[100]{};
          RepairDate tempRepairDate;
          while (true) {
               tempInt = 0;
               for (int i = 0; tempStr[i]; i++) { tempStr[i] =
'\0'; }
               tempRepairDate = RepairDate{};
               switch (editMenu()) {
               case 1:
                    getValidatedString("Enter car brand: ",
tempStr, 0, 3, 24, "Car brand must have length between 3 and 24,
contain letters from a to z and A to Z.\nTry again: ");
                    strcpy_s(record.brand, sizeof(record.brand),
tempStr);
                    break;
               case 2:
                    getValidatedString("Enter car model: ",
tempStr, 1, 2, 24, "Car model must have length between 2 and 24,
contain numbers from 0 to 9, letters from a to z and A to Z and
space.\nTry again: ");
                    strcpy_s(record.model, sizeof(record.model),
tempStr);
                    break;
               case 3:
                    record.repairType = selectRepairType();
                    break;
               case 4:
                    getValidatedInt("Enter repair cost in
dollars: ", tempInt, 1, 1000000, "Repair cost must be a number
between 1 and 1000000.\nTry again: ");
                    record.repairCost = tempInt;
                    break;
```

Продолжение приложения А

```
                case 5:
                     getValidatedInt("Enter mileage in kilometers:
", tempInt, 1, 5000000, "Mileage must be a number between 1 and
5000000.\nTry again: ");
                     record.mileage = tempInt;
                     break;
                case 6:
                     cout << "Enter date of repair:\n";
                     getValidatedInt("Day: ", tempRepairDate.day,
1, 31, "Day must be a number between 1 and 31.\nTry again: ");
                     getValidatedInt("Month: ",
tempRepairDate.month, 1, 12, "Month must be a number between 1
and 12.\nTry again: ");
                     getValidatedInt("Year: ",
tempRepairDate.year, 1900, 2100, "Year must be a number between
1900 and 2100.\nTry again: ");
                     record.repairDate = tempRepairDate;
                     break;
                default:
                     return;
                }
                cout << "Would you like to edit another field?
(Y/N): ";
                char choice = _getch();
                if (choice == 'n' || choice == 'N') {
                     char reportContent[512] = "Edited record:
\n";
                     char recordStr[256];
                     record.toString(recordStr);
                     strcat_s(reportContent,
sizeof(reportContent), recordStr);
                     saveReportToFile(reportContent);
                     break;
                }
                system("cls");
           }
     }

     CarRepairSystem::RepairType
CarRepairSystem::selectRepairType() {
           cout << "Choose repair type: " << endl;
           for (int i = 0; i < AMOUNT_OF_REPAIR_TYPES - 1; i++) {
cout << " " << (i + 1) << ". " << repairTypeStrings[i] << endl;
}
           cout << " Any key to select Other" << endl << "Please
make choice: ";
           int choice;
           cin >> choice;
```

Продолжение приложения А

```
          if (cin.fail()) {
               cin.clear();
               cin.ignore(numeric_limits<streamsize>::max(),
'\n');
               choice = AMOUNT_OF_REPAIR_TYPES;
          }
          return (choice > 0 && choice < AMOUNT_OF_REPAIR_TYPES)
? static_cast<RepairType>(choice - 1) : RepairType::Other;
     }

     CarRepairSystem::RepairRecord*
CarRepairSystem::loadFromFile() const {
          FILE* recordsFile;
          if (fopen_s(&recordsFile, recordsPath, "rb")) {
               cout << "Error opening records file." << endl;
               return nullptr;
          }

          RepairRecord tempRecord;
          RepairRecord* list = new RepairRecord[recordsCount];
          int counter = 0;

          while (fread(&tempRecord, sizeof(RepairRecord), 1,
recordsFile)) {
               list[counter] = tempRecord;
               counter++;
          }

          fclose(recordsFile);

          return list;
     }

     void CarRepairSystem::saveToFile(RepairRecord* list) const {
          FILE* tempFile;
          if (fopen_s(&tempFile, "temp.bin", "wb")) {
               cout << "Error opening temporary file." << endl;
               return;
          }
          for (int i = 0; i < recordsCount; i++) {
fwrite(&list[i], sizeof(RepairRecord), 1, tempFile); }
          fclose(tempFile);
          delete[] list;
          list = nullptr;
          remove(recordsPath);
          if (rename("temp.bin", recordsPath)) { cout << "Error
renaming file." << endl; }
     }
```

Продолжение приложения А

```
     void CarRepairSystem::saveReportToFile(const char* content)
{
          FILE* reportFile;
          if (fopen_s(&reportFile, "report.txt", "a")) {
               cout << "Error opening report file." << endl;
               return;
          }

          time_t now = time(0);
          char timestamp[64];
          ctime_s(timestamp, sizeof(timestamp), &now);
          fprintf(reportFile, "\n=== Report added on: %s",
timestamp);

          if (fprintf(reportFile, "%s", content) < 0) {
               cout << "Error writing to report file." << endl;
          }
          fclose(reportFile);
     }

     void CarRepairSystem::quickSort(RepairRecord* list, int
left, int right) {
          if (left > right) return;
          RepairRecord currentRecord = list[(left + right) / 2];
          int i = left;
          int j = right;
          while (i <= j) {
               while (list[i].repairDate <
currentRecord.repairDate) i++;
               while (list[j].repairDate >
currentRecord.repairDate) j--;
               if (i <= j) {
                    RepairRecord tempRecord = list[i];
                    list[i] = list[j];
                    list[j] = tempRecord;
                    i++;
                    j--;
               }
          }
          quickSort(list, left, j);
          quickSort(list, i, right);
     }

     void CarRepairSystem::selectionSort(RepairRecord* list) {
          RepairRecord tempRecord;
          for (int positionToPut = 0; positionToPut <
recordsCount - 1; positionToPut++) {
               int positionOfMinimum = positionToPut;
```

Продолжение приложения А

```
                for (int j = positionToPut + 1; j < recordsCount;
          j++) {
                     if (list[j].mileage <
list[positionOfMinimum].mileage) positionOfMinimum = j;
                }
                if (positionOfMinimum != positionToPut) {
                     tempRecord = list[positionToPut];
                     list[positionToPut] =
list[positionOfMinimum];
                     list[positionOfMinimum] = tempRecord;
                }
          }
     }

     void CarRepairSystem::insertionSort(RepairRecord* list) {
          RepairRecord tempRecord;
          for (int currentPosition = 1; currentPosition <
recordsCount; currentPosition++) {
                int i = currentPosition;
                while (i > 0 && list[i].repairCost < list[i -
1].repairCost) {
                     tempRecord = list[i];
                     list[i] = list[i - 1];
                     list[i - 1] = tempRecord;
                     i--;
                }
          }
     }

     bool CarRepairSystem::validateStr(char* buffer, int type,
int minLen, int maxLen, const char* errorMsg) {
          cin.getline(buffer, maxLen);
          int strLength = static_cast<int>(strlen(buffer));
          if (!(strLength <= maxLen && strLength >= minLen)) {
cout << errorMsg; return false; }
          for (int i = 0; i < strLength; i++) {
                bool isValidChar = ((buffer[i] >= 'a' && buffer[i]
<= 'z') || (buffer[i] >= 'A' && buffer[i] <= 'Z'));
                if (type == 1) { isValidChar = isValidChar ||
(buffer[i] == ' ') || (buffer[i] >= '0' && buffer[i] <= '9'); }
                if (!isValidChar) {
                     cout << errorMsg;
                     return false;
                }
          }
          return true;
     }
```

Продолжение приложения А

```
     void CarRepairSystem::getValidatedString(const char* prompt,
char* buffer, int type, int minLen, int maxLen, const char*
errorMsg) {
          cout << prompt;
          while (!validateStr(buffer, type, minLen, maxLen,
errorMsg));
     }

     bool CarRepairSystem::validateInt(int& value, int min, int
max, const char* errorMessage) {
          cin >> value;
          if (cin.fail() || !(value <= max && value >= min)) {
               if (cin.fail()) {
                    cin.clear();
                    cin.ignore(numeric_limits<streamsize>::max(),
'\n');
               }
               cout << errorMessage;
               return false;
          }
          return true;
     }
     void CarRepairSystem::getValidatedInt(const char* prompt,
int& value, int min, int max, const char* errorMsg) {
          cout << prompt;
          while (!validateInt(value, min, max, errorMsg));
     }

     CarRepairSystem::RepairRecord::RepairRecord(const char*
brand, const char* model, RepairType repairType, int repairCost,
int mileage, RepairDate repairDate) {
          strcpy_s(this->brand, sizeof(this->brand), brand);
          strcpy_s(this->model, sizeof(this->model), model);
          this->repairType = repairType;
          this->repairCost = repairCost;
          this->mileage = mileage;
          this->repairDate = repairDate;
     }

     void CarRepairSystem::RepairRecord::printRepairRecord(bool
isFirst) const {
          if (isFirst) cout << "---------------------------------
----------\n";
          cout << "Car brand: " << this->brand << "\n"
               << "Car model: " << this->model << "\n"
               << "Repair type: " <<
repairTypeStrings[static_cast<int>(this->repairType)] << "\n"
               << "Repair cost: $" << this->repairCost << "\n"
               << "Mileage: " << this->mileage << "km" << "\n"
               << "Date: " << this->repairDate.day << "." <<
this->repairDate.month << "." << this->repairDate.year << "\n"
```

Продолжение приложения А

```
                << "-----------------------------------------"
                << endl;
     }

     void CarRepairSystem::RepairRecord::toString(char* buffer) {
          sprintf_s(buffer, 256, "%s %s, %s, $%d, %dkm,
%02d.%02d.%04d\n",
                brand, model,
repairTypeStrings[static_cast<int>(repairType)], repairCost,
mileage, repairDate.day, repairDate.month, repairDate.year);
     }

     bool CarRepairSystem::RepairDate::operator>(RepairDate
other) {
          if (year != other.year) return year > other.year;
          if (month != other.month) return month > other.month;
          return day > other.day;
     }

     bool CarRepairSystem::RepairDate::operator<(RepairDate
other) {
          if (year != other.year) return year < other.year;
          if (month != other.month) return month < other.month;
          return day < other.day;
     }

     int main() {
          CarRepairSystem carRepairSystem("data.bin");
          carRepairSystem.start();
          return 0;
     }
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Блок-схема работы программы

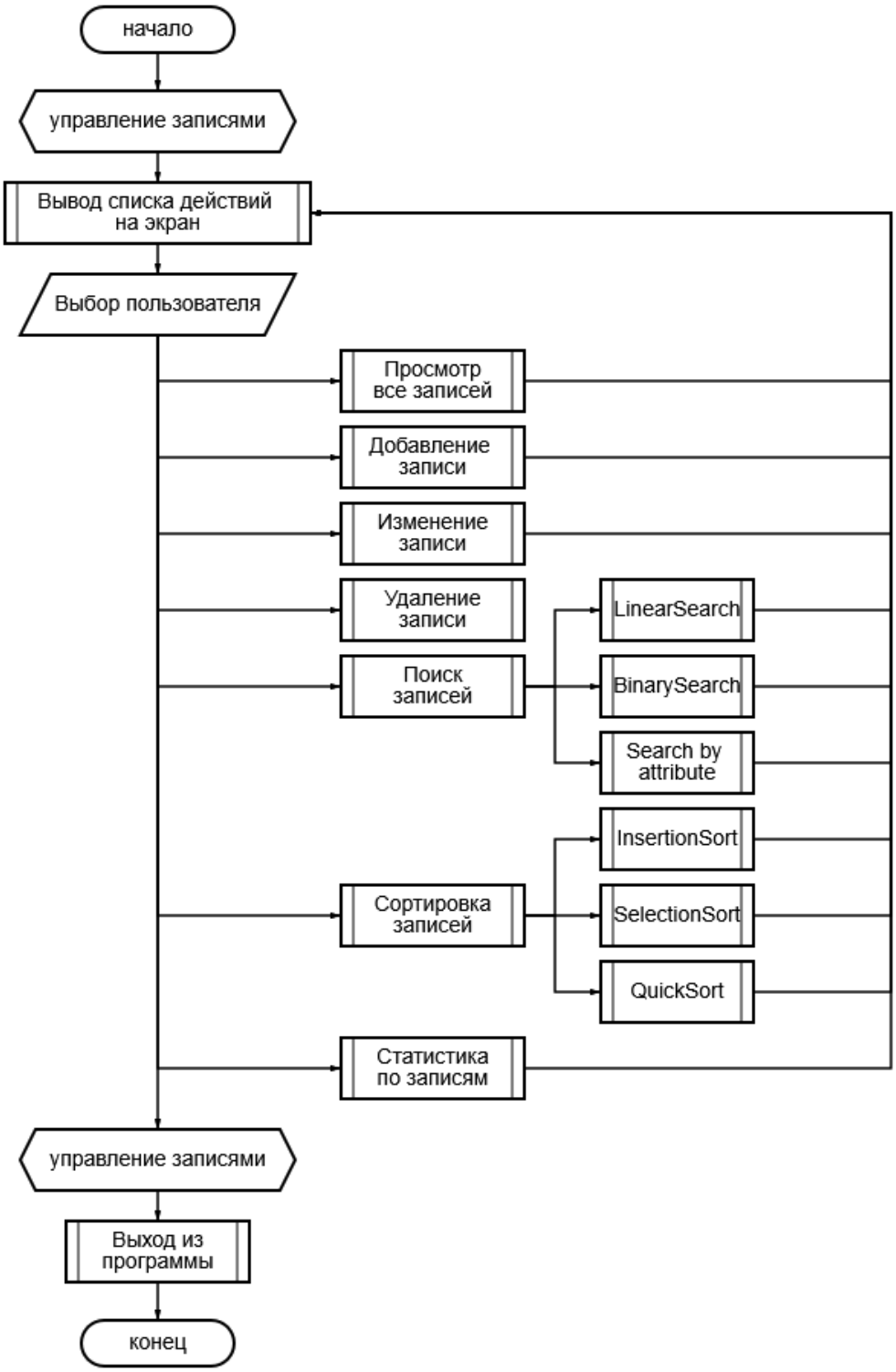

Рисунок Б.1 – Блок-схема работы программы

ВЕДОМОСТЬ ДОКУМЕНТОВ

Обозначение	Наименование	Дополнительные сведения
	Текстовые документы	
БГУИР КП 6-05-0611-03 019 ПЗ	Пояснительная записка	52 с.

					БГУИР КП 6-05-0611-03 019 ВД			
Изм.	Л.	№ докум.	Подп.	Дата	Система учета расходов на ремонт и обслуживание автомобилей	Лит	Лист	Листов
Разраб.		Воинов				У	52	52
Проверил		Новицкая				Кафедра ВМиП гр. 421701		